

LAPORAN TUGAS ALGORITMA DAN STUKTUR DATA
E-COMMERCE ORDER MANAGEMENT & RECOMMENDATION ENGINE
MANAJEMEN PESANAN E-COMMERCE & MESIN REKOMENDASI

Yang diampu oleh Bapak Dr. Eng. Ir. Aji Ery Burhandenny ST., M.AIT.



Disusun Oleh:

- | | |
|--------------------------------|---------------|
| 1. Febri Bayu Nurcahyo | (25051030018) |
| 2. Faiz Nabel Mahendra | (25051030015) |
| 3. Rifan Dwi Pangestu | (25051030020) |
| 4. Muhammad Zidane Wibowo | (25051030021) |
| 5. Abdullah Taqiyyudin Alfatih | (25051030022) |

PROGRAM STUDI TEKNIK ELEKTRO
FAKULTAS TEKNIK UNIVERSITAS NEGERI YOGYAKARTA

2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Platform e-commerce di Indonesia, seperti Tokopedia dan Shopee, saat ini telah menjadi pilar ekonomi digital dengan melayani jutaan transaksi setiap harinya. Skala operasional yang masif ini menuntut sistem manajemen pesanan yang sangat efisien untuk menjaga kepuasan pelanggan, terutama dalam memberikan prioritas layanan kepada pelanggan tingkat Premium.

Terdapat tiga aspek utama yang menjadi pilar dalam keberhasilan operasional sistem e-commerce: manajemen antrian pesanan yang adil dan efisien, katalog produk yang memungkinkan pencarian cepat, serta mesin rekomendasi berbasis pola pembelian (co-purchase) untuk meningkatkan keterlibatan pengguna. Proyek ini merancang simulasi sistem backend e-commerce terintegrasi yang dibangun menggunakan struktur data murni tanpa bantuan pustaka eksternal untuk memahami optimasi performa pada tingkat algoritma.

1.2 Tujuan Sistem

Sistem ini dirancang untuk mencapai beberapa tujuan teknis utama sebagai berikut:

1. **Manajemen Antrian Multi-Prioritas:** Mengelola urutan pemrosesan pesanan secara adil tetapi juga efisien dengan mendahulukan pelanggan berdasarkan tier (Premium, Regular, dan Economy).
2. **Katalog Produk Terstruktur:** Menyediakan mekanisme penyimpanan dan pencarian data produk (kode, harga, stok) yang cepat melalui struktur data pohon biner.
3. **Sistem Rekomendasi Pintar:** Memberikan saran produk yang relevan kepada pelanggan berdasarkan analisis hubungan antar-produk dalam graf konektivitas co-purchase.
4. **Audit dan Riwayat Transaksi:** Mencatat log aktivitas belanja pelanggan secara kronologis dengan fitur pembatalan pesanan terakhir (undo) menggunakan prinsip LIFO.

1.3 Batasan Implementasi

Implementasi sistem ini mengikuti batasan-batasan teknis yang telah ditetapkan:

1. Bahasa pemrograman utama yang digunakan adalah Python 3.
2. Dilarang menggunakan pustaka (library) struktur data bawaan; seluruh struktur data seperti Linked List, Queue, Stack, Binary Search Tree (BST), dan Graph harus diimplementasikan secara mandiri dari nol (from scratch).
3. Antarmuka sistem menggunakan Command Line Interface (CLI) untuk demonstrasi fungsionalitas secara interaktif.
4. Parameter sistem mencakup 100 jenis produk (P001–P100), 50 data pelanggan (C001–C050), dan simulasi sebanyak 300 pesanan acak.

1.4 Diagram Arsitektur Sistem

Sistem ini dibangun atas pipeline yang terdiri dari 6 modul fungsional yang saling berinteraksi secara sinkron:

1. **Multi-Priority Order Queue:** Mengatur antrian masuk berdasarkan 3 tier prioritas menggunakan Linked List.
2. **BST Katalog Produk:** Modul pusat data untuk operasi pencarian, pembaruan stok, dan penyisipan produk baru.
3. **Graph Recommendation Engine:** Menganalisis hubungan antar-produk menggunakan algoritma BFS untuk merekomendasikan produk dalam jangkauan 2-hop.

4. Stack Riwayat Transaksi: Menyimpan log transaksi per pelanggan dan mendukung fungsi undo pesanan.
5. Sorting & Report Module: Mengolah data laporan harian dengan mengurutkan transaksi berdasarkan total harga atau waktu pesan menggunakan algoritma Bubble Sort dan Insertion Sort.
6. CLI E-Commerce: Sebagai pusat kendali yang menghubungkan seluruh modul dengan input pengguna secara interaktif.

BAB II PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data dan Justifikasi

Sistem ini mengintegrasikan empat struktur data utama yang diimplementasikan dari nol (from scratch) tanpa pustaka eksternal untuk menjamin efisiensi operasional pada setiap modulnya.

- a. Multi-tier Queue (Linked List)
Penerapan: Digunakan tiga antrian terpisah untuk tier Premium, Regular, dan Economy.
Justifikasi: Penggunaan Linked List dengan pointer head dan tail memungkinkan operasi enqueue dan dequeue dilakukan dalam waktu konstan $O(1)$. Hal ini sangat krusial dalam sistem e-commerce skala besar untuk memastikan tidak ada hambatan (bottleneck) saat lonjakan pesanan masuk.
- b. Binary Search Tree (BST)
Penerapan: Digunakan untuk mengelola katalog produk dengan kode_produk sebagai kunci (key).
Justifikasi: BST dipilih karena menawarkan efisiensi pencarian, penyisipan, dan penghapusan rata-rata $O(\log n)$. Jika dibandingkan dengan pencarian linear pada array, BST secara signifikan mempercepat validasi stok saat pelanggan melakukan pesanan.
- c. Graph (Adjacency List)
Penerapan: Memodelkan hubungan antar produk berdasarkan pola pembelian bersama (co-purchase).
Justifikasi: Representasi Adjacency List digunakan karena lebih hemat memori dibandingkan Adjacency Matrix, terutama untuk graf yang jarang (sparse graph) di mana tidak semua produk memiliki hubungan pembelian satu sama lain.
- d. Stack (Linked List)
Penerapan: Menyimpan riwayat 10 transaksi terakhir per pelanggan dan mendukung fitur CANCEL_LAST (undo).
Justifikasi: Struktur LIFO (Last-In-First-Out) adalah model matematis yang paling efisien untuk operasi pembatalan, dengan kompleksitas $O(1)$ untuk setiap operasi push dan pop.

2.2 Diagram Interaksi Antar Modul

Sistem bekerja melalui pipeline yang saling berinteraksi sebagai berikut:

1. Modul CLI menerima perintah ORDER.
2. Modul BST Katalog memvalidasi ketersediaan produk dan mengurangi stok.
3. Pesanan dimasukkan ke Multi-Priority Queue sesuai tier pelanggan.
4. Secara sinkron, data transaksi disimpan ke dalam Stack Riwayat pelanggan.

5. Saat perintah SERVE dipanggil, pesanan diproses, dan Modul Graph memberikan rekomendasi produk terkait menggunakan algoritma BFS.
6. Setelah seluruh sesi berakhir, Modul Sorting mengolah data dari antrian yang selesai menjadi laporan harian yang terurut.

2.3 Pseudocode Algoritma Utama (BFS Recommendation)

Algoritma Breadth-First Search (BFS) digunakan untuk mesin rekomendasi guna mencari produk terkait dalam jangkauan maksimal 2-hop pada graf konektivitas produk.

ALGORITHM RekomendasiProduk(graf, produk_awal, max_hop=2):

Initialize queue Q with (produk_awal, 0) {FIFO semantics [14]}

Initialize visited set with {produk_awal}

Initialize list rekomendasi = []

WHILE Q is not empty DO:

current_prod, current_hop = Q.dequeue() [15]

IF current_hop > 0 THEN:

Add current_prod to rekomendasi

IF current_hop < max_hop THEN:

FOR each neighbor in graf.neighbors(current_prod) DO:

IF neighbor not in visited THEN:

Mark neighbor as visited

Q.enqueue((neighbor, current_hop + 1))

RETURN rekomendasi

2.4 Alur Pemrosesan Prioritas (Multi-tier Serve)

Logika pemrosesan pesanan mengikuti aturan ketat di mana pelanggan Premium selalu dilayani terlebih dahulu:

1. Periksa apakah queues["PREMIUM"] tidak kosong. Jika ya, ambil pesanan terdepan.
2. Jika queues["PREMIUM"] kosong, periksa queues["REGULAR"].
3. Jika keduanya kosong, ambil dari queues["ECONOMY"].

Operasi ini memastikan efisiensi layanan tetap berada pada tingkat $O(1)$ tanpa perlu melakukan pengurutan ulang pada antrian setiap kali ada data baru.

BAB III IMPLEMENTASI

3.1 Penjelasan Kode per Modul

Sesuai dengan ketentuan proyek, seluruh struktur data diimplementasikan secara murni tanpa menggunakan library tambahan. Sistem ini dibagi menjadi enam modul utama yang saling berinteraksi dalam pipeline, yaitu:

1. Modul Multi-Priority Order Queue Implementasi menggunakan tiga objek Queue berbasis Linked List yang terpisah untuk tier Premium, Regular, dan Economy. Fungsi `serve_order` menggunakan logika pengecekan berjenjang; sistem akan melakukan `dequeue` pada antrian Premium terlebih dahulu, dan hanya akan beralih ke tier di bawahnya jika antrian di atasnya kosong. Hal ini menjamin efisiensi layanan pada kompleksitas $O(1)$.
2. Modul Katalog Produk (BST) Katalog dikelola melalui kelas `BSTKatalog` di mana setiap node menyimpan data produk dengan `kode_produk` sebagai kunci (key). Fungsi `insert` digunakan untuk membangun pohon katalog di awal sesi, sementara fungsi `search` digunakan untuk validasi stok dan harga secara cepat dengan efisiensi rata-rata $O(\log n)$.
3. Modul Recommendation Engine (Graph) Hubungan antar produk (co-purchase) dimodelkan menggunakan graf tak berarah dengan representasi Adjacency List. Mesin rekomendasi menjalankan algoritma BFS (Breadth-First Search) dengan batasan jangkauan maksimal 2-hop. Saat sebuah pesanan diproses, sistem akan mencari produk tetangga yang memiliki hubungan pembelian terdekat untuk disarankan kepada pelanggan.
4. Modul Riwayat Transaksi (Stack) Setiap pelanggan (C001–C050) memiliki objek Stack pribadi untuk mencatat log belanja mereka. Operasi `push` dilakukan setiap kali pesanan berhasil masuk antrian. Perintah `CANCEL_LAST` memicu operasi `pop` pada Stack tersebut untuk menghapus transaksi terakhir dan mengembalikan stok produk di BST secara otomatis.
5. Modul Sorting Laporan Harian Modul ini mengolah data transaksi yang telah selesai diproses. Digunakan dua algoritma sorting, yaitu Bubble Sort dan Insertion Sort, untuk mengurutkan laporan berdasarkan parameter tertentu (seperti total harga atau waktu). Modul ini juga berfungsi sebagai basis data untuk pengujian eksperimen runtime pada Bab V.
6. Modul CLI Controller Berperan sebagai modul utama (main controller) yang menangani loop interaksi pengguna. Modul ini menerima perintah teks (seperti `ORDER`, `SERVE`, `LAPORAN_ANTRIAN`), melakukan parsing parameter, dan memanggil fungsi-fungsi relevan dari modul struktur data lainnya.

3.2 Screenshot Antarmuka CLI

```
/run/media/faiz/SDCard/tbp-asd-e-commerce-order-management-and-recommendation-engine-kel-3 feat/bst =
o > /usr/bin/python /run/media/faiz/SDCard/tbp-asd-e-commerce-order-management-and-recommendation-engine-kel-3/src/main.py
E-Commerce Order Management Ketik BANTUAN untuk daftar perintah
> BANTUAN

==== DAFTAR PERINTAH CLI E-COMMERCE ====
1. ORDER <cust_id> <prod_id> <tier> <qty>
   -> Memesan produk (Tier: PREMIUM, REGULAR, ECONOMY)
2. SERVE
   -> Melayani order di antrean berdasarkan prioritas tier
3. CANCEL_LAST
   -> Membatalkan pesanan terakhir yang dilayani (pop dari stack global)
4. CARI_PRODUK <kode_produk>
   -> Mencari informasi produk berdasarkan kode
5. UPDATE_STOK <kode_produk> <jumlah_baru>
   -> Mengubah jumlah stok produk tertentu
6. REKOMENDASI <kode_produk>
   -> Menampilkan rekomendasi produk berdasarkan co-purchase
7. RIWAYAT <cust_id>
   -> Menampilkan 10 riwayat pesanan terakhir dari seorang pelanggan
8. LAPORAN_HARIAN
   -> Menampilkan laporan pesanan hari ini yang diurutkan berdasarkan harga
9. UNDO_ORDER <cust_id>
   -> Membatalkan pesanan terakhir dari pelanggan tertentu
10. BANTUAN
    -> Menampilkan daftar perintah ini
11. KELUAR
    -> Keluar dari aplikasi
=====
> |
```

3.3 Contoh Skenario Uji (Input//Output)

Berikut adalah simulasi eksekusi perintah pada sistem berdasarkan parameter 300 order acak:

Skenario 1: Penambahan Pesanan (ORDER)

Input: ORDER C001 P005 PREMIUM 10

Proses: Sistem mencari P005 di BST, mengurangi stok, memasukkan pesanan ke queues["PREMIUM"], lalu mengurangi stok produk P005 di BST.

Output:

Big-O: $O(\log n)$ untuk BST Search + $O(1)$ untuk Queue Enqueue

[✓] Order Berhasil Dibuat!

ID: 0 | C001 memesan 10x Mouse Model-5

Total: Rp 46.950.000 (PREMIUM)

Skenario 2: Pemrosesan Pesanan (SERVE)

Input: SERVE

Proses: Mengambil pesanan terdepan dari antrian tertinggi (Premium), mencari data produk, menghapus order dari queue order lalu menambahkan ke stack riwayat

Output:

Big-O: $O(1)$ untuk Dequeue

MELAYANI PESANAN

Order ID : 0
Pelanggan : C001
Tier : PREMIUM
Produk : P005 - Mouse Model-5
Jumlah : 10
Total Bayar : Rp 46.950.000

Skenario 3: Pembatalan Transaksi (CANCEL_LAST)

Input: CANCEL_LAST

Proses: Melakukan pop pada stack riwayat pembelian dan memperbarui stok di BST.

Output:

Big-O: $O(1)$ Stack Pop + $O(\log n)$ update stok BST

[!] CANCEL BERHASIL: Pesanan #0 dibatalkan.

Stok P005 telah dikembalikan (+10).

BAB IV ANALISIS BIG-O

4.1 Analisis Kompleksitas Waktu

Berdasarkan implementasi sistem, berikut adalah analisis kompleksitas waktu untuk operasi utama pada setiap modul menggunakan notasi "Big-O".

1. Multi-Priority Order Queue (Linked List): Operasi ORDER (enqueue) dan SERVE (dequeue) masing-masing memiliki kompleksitas $O(1)$. Hal ini dicapai karena penggunaan Linked List manual dengan pointer head dan tail yang memungkinkan penambahan elemen di akhir dan pengambilan di awal tanpa harus menggeser elemen lain (shifting). Sistem ini tetap efisien meskipun menggunakan tiga tier antrian karena hanya memerlukan pengecekan berjenjang yang bersifat konstan.
2. BST Katalog Produk: Operasi pencarian (search) dan penyisipan (insert) produk memiliki kompleksitas rata-rata $O(\log n)$. Pada struktur pohon biner, setiap keputusan pencarian memangkas setengah dari sisa kandidat node yang ada. Namun, perlu dicatat bahwa dalam skenario terburuk (misal data dimasukkan secara berurutan), tinggi pohon bisa menjadi linear sehingga kompleksitasnya berubah menjadi $O(n)$.
3. Graph Recommendation Engine (BFS): Mesin rekomendasi menggunakan algoritma Breadth-First Search (BFS) untuk mencari produk terkait dalam jangkauan 2-hop. Kompleksitas waktunya adalah $O(V+E)$, di mana V adalah jumlah produk (vertex) dan E adalah jumlah hubungan co-purchase (edge). Penggunaan adjacency list memastikan sistem hanya mengunjungi tetangga yang relevan tanpa memeriksa seluruh matriks.
4. Stack Riwayat Transaksi: Operasi push untuk menyimpan riwayat dan pop untuk pembatalan (CANCEL_LAST) berjalan dalam waktu konstan $O(1)$. Karena operasi hanya

dilakukan pada puncak (top) struktur data, tidak ada iterasi yang bergantung pada jumlah data pelanggan.

5. Modul Sorting Laporan: Algoritma Bubble Sort dan Insertion Sort yang digunakan untuk mengurutkan laporan harian memiliki kompleksitas waktu $O(n^2)$ dalam skenario rata-rata dan terburuk. Hal ini dikarenakan adanya nested loop di mana setiap elemen dibandingkan dengan elemen lainnya.

4.2 Analisis Kompleksitas Ruang

Seluruh struktur data diimplementasikan untuk mengoptimalkan penggunaan memori:

1. Queue & Stack: Memerlukan ruang $O(n)$ sebanding dengan jumlah pesanan atau riwayat yang disimpan.
2. BST: Memerlukan $O(n)$ untuk menyimpan n produk di katalog.
3. Graph: Menggunakan representasi adjacency list yang efisien di ruang $O(V+E)$, sangat cocok untuk graf e-commerce yang cenderung jarang (sparse).

4.3 Tabel Ringkasan Big-O

Berikut adalah ringkasan performa algoritma yang diimplementasikan:

Modul / Operasi	Struktur Data	Waktu (Rata-rata)	Ruang
ORDER / SERVE	Multi-tier Queue	$O(1)$	$O(n)$
CARI_PRODUK	BST	$O(\log n)$	$O(n)$
REKOMENDASI	Graph (BFS)	$O(V+E)$	$O(V)$
CANCEL_LAST	Stack	$O(1)$	$O(1)$ per item
LAPORAN_HARIAN	Bubble / Insertion	$O(n^2)$	$O(1)$ (in-place)

4.4 Perbandingan Teoritis vs Eksperimen

Secara teoritis, operasi pada antrian dan riwayat harus menunjukkan waktu eksekusi yang hampir identik terlepas dari jumlah data ($O(1)$). Sebaliknya, untuk modul sorting, peningkatan ukuran dataset dari $N=50$ ke $N=300$ diperkirakan akan meningkatkan waktu eksekusi secara kuadratik. Analisis lebih lanjut mengenai data hasil uji nyata akan dibahas pada Bab V: Hasil Eksperimen untuk memvalidasi apakah performa program di lingkungan Python sesuai dengan prediksi Big-O ini.

BAB V HASIL EKSPERIMEN

5.1 Prosedur Pengujian

Eksperimen dilakukan untuk memvalidasi efisiensi algoritma sorting yang diimplementasikan pada modul laporan harian. Fokus pengujian adalah membandingkan Bubble Sort dan Insertion Sort yang keduanya diimplementasikan secara mandiri pada struktur data Linked List. Parameter pengujian yang ditetapkan adalah sebagai berikut:

1. Ukuran Dataset (N): 50, 100, dan 300 data pesanan.

2. Kondisi Data: Data dihasilkan secara acak untuk mensimulasikan beban kerja nyata, dengan penggunaan random seed untuk memastikan konsistensi hasil antar percobaan.
3. Metrik: Waktu eksekusi diukur dalam satuan secon menggunakan modul time pada Python.

5.2 Tabel Hasil Runtime Sorting

Tabel berikut menyajikan perbandingan waktu eksekusi antara kedua algoritma berdasarkan hasil simulasi pada lingkungan eksekusi program:

Ukuran Data (N), Bubble Sort (ms), Insertion Sort (ms)

Ukuran Data (N)	Bubble Sort (s)	Insertion Sort (s)
50 Pesanan	0.0003	0.0002
100 Pesanan	0.0011	0.0007
300 Pesanan	0.0139	0.0116
3000 pesanan	0.8414	0.3809

5.3 Analisis dan Diskusi

Berdasarkan data eksperimen di atas, dapat ditarik beberapa poin analisis sebagai berikut:

1. Validasi Kompleksitas $O(n^2)$: Seiring dengan peningkatan ukuran dataset (dari $N=50$ ke $N=3000$), waktu eksekusi meningkat secara kuadratik. Hal ini mengonfirmasi prediksi teoritis Big-O di mana algoritma dengan nested loop akan mengalami degradasi performa yang signifikan saat jumlah elemen bertambah besar.
2. Efisiensi Algoritma pada Data Acak: Pada dataset acak, Insertion Sort secara konsisten menunjukkan waktu eksekusi yang lebih rendah dibandingkan Bubble Sort. Hal ini disebabkan karena Insertion Sort memproses elemen dengan memindahkan elemen yang belum terurut ke posisi yang tepat tanpa harus melakukan iterasi penuh pada seluruh daftar jika kondisi sudah terpenuhi.
3. Kasus Data 'Hampir Terurut': Dalam operasional e-commerce harian, seringkali pesanan baru ditambahkan di akhir daftar yang sudah rapi. Secara teoritis dan didukung hasil uji, Insertion Sort jauh lebih efisien untuk kondisi data 'hampir terurut' dengan kompleksitas mendekati $O(n)$, sementara Bubble Sort tetap melakukan perbandingan elemen yang tidak perlu kecuali jika dioptimasi dengan bendera swap.
4. Refleksi Desain: Meskipun kedua algoritma ini memadai untuk laporan skala kecil, untuk sistem skala industri seperti Tokopedia atau Shopee dengan jutaan transaksi, penggunaan algoritma $O(n^2)$ tidak lagi layak dan perlu digantikan dengan algoritma $O(n \log n)$ seperti Merge Sort yang lebih stabil pada Linked List.

BAB VI JAWABAN PERTANYAAN ANALISIS

6.1 Analisis Efisiensi Multi-tier Queue vs Priority Queue Tunggal

Pertanyaan: Bandingkan efisiensi antara Multi-tier Queue (3 antrian terpisah) dengan satu Priority Queue tunggal (berbasis Linked List terurut) untuk throughput 300 order/hari.

Analisis:

- Multi-tier Queue: Operasi ENQUEUE dilakukan pada tier yang relevan dengan kompleksitas $O(1)$, dan SERVE (dequeue) juga $O(1)$ karena sistem hanya memeriksa tiga head secara berjenjang. Untuk 300 order, total operasi adalah $300+300=600$ operasi dasar.
- Priority Queue (Sorted Linked List): Operasi SERVE adalah $O(1)$, namun ENQUEUE memerlukan pencarian posisi yang tepat dengan kompleksitas $O(n)$. Secara akumulatif, total operasi untuk 300 order mengikuti deret aritmatika $\sum_{i=1}^{300} i \approx 23002 = 45.000$ operasi.

Kesimpulan: Multi-tier Queue jauh lebih efisien untuk throughput tinggi. Dengan 60% pelanggan Regular, Multi-tier menghindari degradasi performa $O(n)$ pada saat penambahan pesanan baru.

6.2 Estimasi Eksplorasi Node pada BFS Rekomendasi

Pertanyaan: Jika setiap produk rata-rata memiliki 5 hubungan co-purchase dan graf berisi 100 produk, estimasikan node yang dijelajahi BFS dalam 2 hop.

Analisis:

Menggunakan analisis pohon ekspansi:

Level 0 (Sumber): 1 node (produk yang baru dibeli).

Level 1 (1-hop): $1 \times 5 = 5$ node tetangga langsung.

Level 2 (2-hop): $5 \times 5 = 25$ node tetangga dari tetangga.

Total Estimasi: $1+5+25=31$ node yang dijelajahi.

Perbandingan: Jika menggunakan brute-force $O(V^2)$ untuk memeriksa seluruh kemungkinan pasangan pada 100 produk, diperlukan $100^2=10.000$ operasi. BFS membatasi pencarian hanya pada produk yang relevan secara topologis, sehingga jauh lebih cepat untuk mesin rekomendasi real-time.

6.3 Circular Buffer vs Stack Linked List untuk Riwayat Transaksi

Pertanyaan: Jelaskan kelebihan circular buffer berbasis array dibandingkan Stack Linked List untuk menyimpan 10 riwayat terakhir.

Analisis:

Akses Elemen: Circular buffer (array) memungkinkan akses elemen ke- k dalam waktu konstan $O(1)$ menggunakan indeks. Linked List memerlukan traversi $O(k)$ untuk mencapai posisi tertentu.

Memori: Circular buffer menggunakan memori statis yang teralokasi sejak awal, menghindari overhead pembuatan objek Node baru dan pointer next yang memakan memori ekstra (setiap node Linked List memerlukan 2 referensi: data dan pointer).

Kesimpulan: Untuk kapasitas tetap (10 item), circular buffer lebih unggul karena efisiensi ruang dan kecepatan akses acak (random access).

6.4 Efisiensi Sorting pada Data 'Hampir Terurut'

Pertanyaan: Identifikasi algoritma sorting yang lebih baik untuk data laporan harian yang bersifat 'hampir terurut'.

Analisis:

Meskipun keduanya memiliki Big-O $O(n^2)$, Insertion Sort jauh lebih cepat daripada Bubble Sort pada data hampir terurut.

Alasan: Insertion Sort hanya memerlukan satu perbandingan per elemen jika elemen tersebut sudah berada di posisi yang benar (mendekati $O(n)$ pada skenario terbaik), sedangkan Bubble Sort tetap melakukan iterasi besar kecuali dioptimasi secara manual dengan bendera swap. Hasil eksperimen pada Bab V mengonfirmasi bahwa waktu eksekusi Insertion Sort pada dataset pesanan harian selalu lebih rendah.

6.5 Strategi Scaling Graf ke 1 Juta Node

Pertanyaan: Bagaimana modifikasi adjacency list jika jumlah produk mencapai 1 juta node seperti pada platform Tokopedia?

Analisis:

Modifikasi Struktur: Mengganti adjacency list standar (list of lists) menjadi Adjacency Map (menggunakan hash map untuk menyimpan tetangga).

Trade-off:

Waktu Akses: Mengecek apakah dua produk memiliki hubungan co-purchase menjadi $O(1)$ rata-rata, dibandingkan $O(\text{degree})$ pada list biasa.

Memori: Penggunaan memori akan meningkat karena overhead dari struktur hash map, namun tetap di jangkauan $O(V+E)$.

Kompleksitas: Implementasi lebih rumit namun sangat diperlukan untuk sistem skala masif guna menghindari bottleneck pencarian pada produk populer (high-degree nodes).

BAB VII KESIMPULAN

7.1 Ringkasan Pencapaian

Proyek ini telah berhasil merancang dan mengimplementasikan simulasi sistem backend e-commerce yang mengintegrasikan empat struktur data utama (Queue, BST, Graph, dan Stack) ke dalam satu pipeline operasional yang sinkron. Sistem ini mampu mengelola antrian pesanan berdasarkan tier pelanggan (Premium, Regular, Economy) dengan efisiensi waktu konstan, melakukan pencarian produk secara cepat melalui katalog pohon biner, serta memberikan rekomendasi produk berbasis pola pembelian menggunakan algoritma BFS. Seluruh modul fungsional telah diuji secara interaktif melalui antarmuka CLI dan terbukti berjalan sesuai spesifikasi teknis yang ditetapkan.

7.2 Refleksi Desain dan Trade-off

Berdasarkan hasil analisis dan eksperimen, terdapat beberapa poin refleksi mengenai pilihan desain struktur data yang diambil:

Efisiensi Antrian: Penggunaan Multi-tier Queue terbukti jauh lebih efisien untuk throughput pesanan yang tinggi dibandingkan satu Priority Queue tunggal, karena menghindari operasi penyisipan terurut $O(n)$ setiap kali pesanan baru masuk.

Kecepatan vs Skalabilitas: BST menawarkan kecepatan pencarian rata-rata $O(\log n)$ yang sangat baik untuk katalog, namun sistem ini rentan terhadap degradasi performa menjadi $O(n)$ jika data produk dimasukkan secara berurutan leksikografis.

Manajemen Memori: Representasi graf menggunakan Adjacency List memberikan efisiensi ruang yang optimal untuk memodelkan hubungan co-purchase antar produk yang cenderung jarang (sparse graph).

Batas Algoritma Sorting: Eksperimen runtime menunjukkan bahwa algoritma Insertion Sort dan Bubble Sort ($O(n^2)$) cukup untuk laporan skala kecil, namun menjadi titik hambat (bottleneck) utama ketika jumlah pesanan mencapai skala industri.

7.3 Saran Pengembangan

Untuk meningkatkan sistem ini ke tingkat produksi yang sesungguhnya, beberapa langkah pengembangan berikut disarankan:

Optimasi Skalabilitas: Memodifikasi representasi graf menjadi Adjacency Map (menggunakan hash map) untuk mempercepat pengecekan hubungan antar produk pada skala jutaan node.

Peningkatan Algoritma: Mengganti algoritma pengurutan laporan harian menjadi Merge Sort yang menjamin kompleksitas $O(n \log n)$ demi stabilitas performa pada dataset besar.

Penyeimbangan Pohon: Mengimplementasikan struktur Self-Balancing BST seperti AVL Tree atau Red-Black Tree untuk menjamin tinggi pohon tetap logaritmik meskipun data dimasukkan secara terurut.

Fitur Undo Tingkat Lanjut: Mengganti Stack Linked List dengan Circular Buffer berbasis array untuk riwayat transaksi guna mengoptimalkan penggunaan memori dan kecepatan akses elemen.